AutoMLOps: A Fully Automated End-to-End Machine Learning Pipeline with MLflow Experiment Tracking and Docker Container Orchestration

Darshit Paghdar
23it069@charusat.edu.in
K.D. Patel Department of Information Technology,
Chandubhai S. Patel Institute of Technology (CSPIT),
Charotar University of Science and Technology (CHARUSAT),
Changa, Gujarat, India

Abstract—The transition of machine learning models from experimental prototypes to production-ready systems remains one of the most persistent and costly challenges in applied AI. Industry research consistently shows that the overwhelming majority of developed models never reach deployment, primarily due to fragmented tooling, manual processes, and the absence of a unified operational framework. This paper introduces AutoMLOps, a modular, domain-agnostic Machine Learning Operations (MLOps) framework engineered to eliminate manual intervention across every phase of the ML lifecycle—spanning raw data ingestion, preprocessing, exploratory analysis, unsupervised clustering, supervised classification, and model deployment. The framework is organized into six sequentially executed stages, each coordinated by a central orchestration controller. Every stage automatically records hyperparameters, evaluation metrics, and output artifacts into MLflow's experiment tracking system without human involvement. Validated on a structured transactional benchmark dataset, the system achieves a Random Forest classification accuracy of 92.51%, a ROC-AUC score of 0.9642, and a K-Means Silhouette Score of 0.68, completing the entire pipeline in approximately 141 seconds on commodity hardware. The paper additionally presents a planned four-container Docker architecture designed to encapsulate training, serving, experiment tracking, and automated monitoring into a single unit deployable on any cloud provider with a single command.

Keywords—*MLOps, Machine Learning Pipeline Automation, Experiment Tracking, MLflow, Docker, Model Deployment, Model Monitoring, Feature Engineering, RFM Analysis, K-Means Clustering, Random Forest Classification, CI/CD for ML, Reproducibility*

I. Introduction

A. Problem Statement

The deployment of machine learning models into production environments continues to be one of the most persistent unsolved problems in applied artificial intelligence. Foundational research by Sculley et al. [1] demonstrated that in a typical real-world ML system, actual model code represents only a small fraction of the total codebase—the remainder is consumed by data pipelines, serving infrastructure, configuration management, and monitoring logic. This hidden technical debt translates directly into operational failure rates: industry surveys report that 87% of machine learning projects never successfully transition from prototype to production [10].

The root causes of this failure rate are well documented. Training and production environments frequently diverge, producing silent reproducibility failures. Model deployment typically remains a labor-intensive process that resists reliable repetition. Once deployed, models degrade as real-world data distributions shift over time, often without any automated mechanism to detect or respond to this change. Experimental results are stored inconsistently across team members, making systematic comparison and auditability difficult or impossible. These challenges are universal and apply regardless of the specific application domain.

B. Motivation

Machine Learning Operations (MLOps) has emerged as a response to these challenges, applying DevOps principles—automation, versioning, continuous integration, and systematic monitoring—to the ML development lifecycle. However, most currently available MLOps implementations fall into one of three limiting categories: cloud-vendor-locked solutions that create provider dependency (such as AWS SageMaker Pipelines or Google Vertex AI); heavyweight orchestration frameworks demanding significant infrastructure expertise (such as Kubeflow or Apache Airflow); or narrowly scoped tools addressing only a single lifecycle aspect without providing an integrated solution.

A clear gap exists for a lightweight, open-source, vendor-agnostic MLOps system that practitioners can execute locally on standard hardware and deploy to any cloud provider without code modification. AutoMLOps is designed specifically to address this gap.

C. Key Contributions

This paper makes five primary contributions:

- **A modular six-stage automated ML pipeline** implemented in Python, with sequential execution, structured error handling, per-stage timing, and strict separation of concerns between pipeline stages.
- **Zero-touch MLflow integration** where every pipeline stage independently logs all hyperparameters, evaluation metrics, and output artifacts without any manual configuration or intervention required.
- **A domain-agnostic architectural design** making no assumptions about industry, dataset structure, or problem type. The benchmark dataset used in this work serves purely as a controlled testing vehicle; the methodology transfers to any structured tabular dataset.

- **A complete Docker containerization design** comprising a four-service architecture covering ML training, REST inference serving, experiment tracking, and automated monitoring, enabling one-command deployment to any cloud provider.
- **A monitoring-ready system** where clustering and classification quality metrics are tracked across pipeline runs, forming the quantitative foundation for future automated drift detection and retraining triggers.

D. Paper Organization

Section II surveys related work. Section III presents the overall system architecture. Section IV details the six pipeline stages. Section V describes MLflow integration. Section VI outlines the Docker containerization plan. Section VII reports experimental results. Section VIII covers monitoring design. Section IX discusses limitations. Section X concludes the paper.

II. Related Work

A. MLOps Platforms and Frameworks

Zaharia et al. [2] introduced MLflow as an open-source platform for managing the complete ML lifecycle through four components: Tracking, Projects, Models, and Registry. Since its release, MLflow has achieved widespread adoption in the open-source ML community. Kubeflow provides Kubernetes-native ML pipeline orchestration, enabling complex DAG-based workflows across distributed compute clusters. Apache Airflow, originally built at Airbnb, offers general-purpose workflow orchestration and has been widely applied to ML pipelines. Netflix's Metaflow provides a Python-native API oriented around data scientist workflows.

Cloud-native solutions including AWS SageMaker Pipelines, Google Vertex AI, and Azure Machine Learning Pipelines offer managed end-to-end MLOps tightly integrated with their cloud ecosystems. Their primary limitation is vendor dependency: models, artifacts, and infrastructure become tightly coupled to a single provider. AutoMLOps distinguishes itself by prioritizing local-first execution, open-source tooling, minimal configuration, and a cloud-provider-agnostic deployment model.

B. Automated Machine Learning

AutoML frameworks such as TPOT [3], Auto-sklearn [4], and H2O AutoML automate model selection and hyperparameter optimization within a single modeling stage. AutoMLOps operates at a different level of abstraction—it automates the complete data science workflow across multiple stages including ingestion, preprocessing, analysis, training, evaluation, and deployment readiness. The two approaches are orthogonal and complementary; an AutoML step could be embedded within the AutoMLOps training stage as a future enhancement.

C. Containerized Scientific Computing

Boettiger [5] established the value of Docker for reproducible scientific computing, showing that containerization resolves the classic "works on my machine" problem by packaging the full execution environment alongside application code. Multi-container ML architectures using Docker Compose have since become a standard pattern in production ML engineering, enabling co-deployment of training, serving, and monitoring services on a single host or across distributed infrastructure.

III. System Architecture

A. Layered Design

AutoMLOps is structured across four distinct architectural layers, each carrying a clearly defined responsibility that prevents coupling between concerns:

B. Technology Stack

Table I summarizes the complete technology stack across all system layers.

Component	Technology	Purpose
Core Language	Python 3.x	All pipeline, preprocessing, and modeling logic
Pipeline Orchestrator	Custom run_pipeline.py	Sequential stage execution and reporting
Machine Learning	scikit-learn	K-Means clustering, Random Forest classification
Data Processing	pandas, NumPy	Data manipulation, aggregation, feature engineering
Feature Scaling	StandardScaler	Normalization for distance-sensitive algorithms
Visualization	Matplotlib, Seaborn	Automated programmatic chart generation
Experiment Tracking	MLflow	Tracking of params, metrics, and artifacts
Metadata Backend	SQLite (mlflow.db)	Persistent file-based experiment database
Future: Containers	Docker + Docker Compose	Service isolation and deployment portability
Future: Inference API	Flask / FastAPI	REST endpoint for real-time model serving
Future: Monitoring	MLflow + Statistical Tests	Automated drift detection and retraining

Table I. AutoMLOps Technology Stack

IV. Six-Stage Pipeline Design

A. Stage 1 — Data Ingestion and Quality Assessment

Script: *check_missing_values.py*

The first stage loads raw data from a configurable CSV source and generates a comprehensive data quality baseline before any transformation is applied. Operations include dataset shape computation, per-column missing value counting and percentage calculation, statistical profiling across all numeric features (minimum, maximum, mean,

standard deviation, and quartiles), and exact duplicate row detection. All findings are logged to MLflow as structured metrics for numerical comparison and as human-readable artifact files. The stage produces no data modifications and outputs solely an analytical report under the MLflow run Step1_Data_Exploration.

B. Stage 2 — Data Preprocessing and Feature Engineering

Script: *data_preprocessing.py*

The second stage converts the raw input into a clean, analysis-ready feature matrix through three operation classes. Data cleaning removes records with absent categorical descriptions, retaining over 85% of the dataset. Rather than discarding records with missing entity identifiers—which would eliminate approximately 25% of records—surrogate identifiers are programmatically generated to preserve all behavioral signal. Duplicate records are then removed.

Temporal decomposition extracts year, month, day, hour, and day-of-week components from invoice timestamps, enabling discovery of seasonality and peak activity patterns. Transaction-level engineered features include TotalPrice (quantity multiplied by unit price), TransactionSize (total quantity per invoice), and UniqueItems (distinct product count per invoice). Entity-level aggregated features include AvgBasketValue, AvgBasketSize, Volatility, FavoriteDay, and Country. Categorical features receive one-hot encoding; ordinal features receive label encoding. All numeric features are standardized to zero-mean and unit-variance using StandardScaler, which is critical for the distance-sensitive K-Means algorithm downstream.

C. Stage 3 — Exploratory Data Analysis

Script: *eda.py*

The third stage produces ten analytical visualizations entirely through code: temporal revenue trends, geographic revenue concentration, product demand distribution, activity heatmaps by hour and day-of-week, entity value distribution and skewness, engagement frequency histograms, growth trend analysis, and concentration risk profiling for high-value entities. Each chart is automatically logged as a PNG artifact to the MLflow run Step3_EDA, ensuring that every training run preserves its own complete set of exploratory visualizations for cross-run comparison.

D. Stage 4 — Unsupervised Clustering with RFM Analysis

Script: *train_model.py*

The fourth stage partitions the entity population into behaviorally homogeneous groups using K-Means clustering on a three-dimensional RFM feature space. Recency (R) quantifies days elapsed since the last transaction; lower values reflect more current engagement. Frequency (F) counts the total number of distinct transactions; higher values indicate deeper engagement. Monetary (M) captures cumulative spending; higher values denote greater economic contribution.

K-Means is configured with k-means++ centroid initialization. Optimal k is determined automatically through two independent criteria: the Elbow Method (inflection point of the Within-Cluster Sum of Squares curve) and Silhouette Score Maximization (selection of k yielding the highest mean silhouette coefficient). Both methods independently converge on k=4 for the validation dataset. Table II presents clustering validation results.

Validation Metric	Score	Interpretation
Silhouette Score [8]	0.68	Good separation; exceeds 0.50 acceptable threshold
Davies-Bouldin Index [9]	Low	Clusters are well-separated relative to scatter
Calinski-Harabasz Score	High	High ratio of between-cluster to within-cluster variance

Table II. Stage 4 Clustering Validation Metrics

E. Stage 5 — Cluster Labeling and Segment Interpretation

Script: *analyze_clusters.py*

The fifth stage translates raw cluster identifiers into semantically meaningful operational labels. Each cluster is independently ranked along each RFM dimension, and composite score profiles are mapped to labels through deterministic rules: entities scoring high across all three dimensions are classified as Champions; medium-to-high composites become Potential Loyalists; clusters with long inactivity and minimal frequency and spend are marked At Risk; those with very high recency and near-zero engagement are designated Lost. Per-segment statistics covering entity count, revenue share, and estimated churn probability are computed and logged alongside four visualizations to the MLflow run Step5_Cluster_Labeling.

F. Stage 6 — Supervised Classification with Random Forest

Script: *train_classifier.py*

The sixth stage trains a supervised Random Forest classifier using the segment labels generated by Stage 5 as training targets. This stage is architecturally essential for production use: K-Means requires the full dataset at inference time, making it impractical for real-time single-record scoring. The Random Forest classifier learns a direct mapping from behavioral features to segment labels, enabling sub-millisecond inference on individual new records without requiring cluster recomputation.

Five features are used: AvgBasketValue, AvgBasketSize, Volatility, FavoriteDay (encoded), and Country (encoded). The model is configured with 100 estimators, Gini impurity, and sqrt(n_features) subsampling. An 80/20 stratified train-test split with random_state=42 ensures full reproducibility. Table III reports classification results on the held-out test set.

Performance Metric	Score
Accuracy	92.51%
Precision (weighted average)	92.66%
Recall (weighted average)	92.51%

Performance Metric	Score
F1 Score (weighted average)	92.47%
ROC-AUC (one-vs-rest, multi-class)	0.9642
Single-record inference latency	< 1 ms

Table III. Stage 6 Random Forest Classification Results

Automated feature importance extraction reveals that AvgBasketValue accounts for 91.25% of the model's discriminating power. This confirms that average spend per transaction is the single strongest behavioral predictor of segment membership regardless of geography or temporal preferences.

V. MLFLOW EXPERIMENT TRACKING INTEGRATION

A. Architecture

MLflow is embedded as a first-class system component, not an optional add-on. The tracking configuration uses a file-backed SQLite database (mlflow.db) as the backend metadata store and the local filesystem (./mlartifacts/) as the artifact repository. This setup requires no external server during development while providing a clear upgrade path to a remote tracking server with object storage for production use. A single named experiment groups all six pipeline runs, each following the StepN_StageName naming convention for unambiguous identification in the MLflow UI. Across the complete pipeline execution, more than 40 parameters, 50 metrics, and 30 artifacts are automatically recorded.

B. Cross-Stage Tracking Summary

Table IV summarizes the parameters, metrics, and artifacts logged at each pipeline stage.

Run	Stage	Metrics Logged	Key Artifacts
1	Data Exploration	Missing counts, duplicates	missing_values.csv, data_types.json
2	Preprocessing	Row retention rate	processed_sample.csv, preprocessing_report.txt
3	EDA	N/A	10 PNG visualization charts
4	K-Means Clustering	Silhouette, Davies-Bouldin, Calinski-Harabasz	elbow_curve.png, cluster_summary.csv
5	Cluster Labeling	Revenue share, churn probability per segment	4 PNG charts, segment_analysis_report.txt
6	Random Forest	Accuracy, Precision, Recall, F1, ROC-AUC	confusion_matrix.png, roc_curves.png, trained model

Table IV. MLflow Cross-Stage Tracking Summary

C. Model Registry

The trained Random Forest classifier is registered in the MLflow Model Registry under the name SegmentClassifier. This enables version tracking across retraining cycles and staged promotion from None through Staging to Production to Archived status. The standard promotion workflow enforces quality gates at every transition, ensuring that the prior Production model is always preserved as an Archived version to support instant rollback.

VI. DOCKER CONTAINERIZATION ARCHITECTURE (PLANNED)

A. Design Goals

The Docker containerization phase—the system's immediate next development milestone—targets five objectives: complete environment reproducibility across machines and operating systems; service isolation with each component in its own container; one-command startup through docker compose up; cloud-provider-agnostic deployment with zero application code modifications; and independent horizontal scaling of the inference service.

B. Four-Container Design

Container 1 — mlflow-server: Hosts the MLflow tracking server on port 5000 with persistent volumes for the artifact directory and SQLite database. All other containers connect via the internal Docker network.

Container 2 — pipeline-runner: Packages all six Python pipeline scripts into a single image. Connects to mlflow-server via MLFLOW_TRACKING_URI and executes run_pipeline.py as the default command with a declared dependency on mlflow-server availability.

Container 3 — api-server: A Flask or FastAPI service that loads the Production-stage model from the MLflow Model Registry at startup and exposes GET /health and POST /predict HTTP endpoints on port 8080 for real-time single-record inference.

Container 4 — monitor: A scheduling container running on a configurable cron schedule. On each cycle it retrieves the latest MLflow run metrics, computes a 7-day rolling accuracy window, executes Kolmogorov-Smirnov and Chi-squared drift tests, and triggers automatic pipeline retraining when accuracy falls below the configured threshold (default: 90%).

C. Cloud Deployment Pathway

The identical Docker images deploy without modification to any major cloud provider: AWS via ECR and ECS Fargate or EKS; Google Cloud via Artifact Registry and Cloud Run or GKE; and Azure via Azure Container Registry and Container Instances or AKS. No code changes are required between local Docker Compose execution and any cloud deployment

target, which constitutes the system's primary portability guarantee.

VII. Experimental Results

A. Validation Dataset

The UCI Online Retail Dataset (2015) was used as a controlled testing and validation environment. This publicly available structured transactional dataset was selected for its structural complexity and scale rather than its domain relevance. The AutoMLOps pipeline is designed to process any structured tabular dataset; the UCI dataset functions as a representative test vehicle demonstrating system capability on a non-trivial real-world input. The raw dataset contains 541,909 records across 8 columns representing 242,065 unique entities across 38 geographic regions over a 12-month period.

B. Pipeline Execution Performance

Table V reports execution time and artifact output for each pipeline stage running on standard commodity hardware (4-core CPU, 8 GB RAM, no GPU required).

Stage	Script	Execution Time	Output Artifacts
1	check_missing_values.py	~12 seconds	2 files
2	data_preprocessing.py	~8 seconds	2 files
3	eda.py	~22 seconds	10 PNG charts
4	train_model.py	~45 seconds	4 charts + 1 CSV
5	analyze_clusters.py	~19 seconds	4 charts + 1 TXT
6	train_classifier.py	~35 seconds	3 charts + model + 2 files
Total	run_pipeline.py	~141 seconds	30+ artifacts

Table V. Pipeline Stage Execution Performance on Standard Hardware

VIII. Monitoring and Continuous Learning

A. Current Passive Monitoring

The system currently supports passive monitoring through MLflow metric comparison across successive pipeline runs. Three drift signal categories are automatically tracked. Data quality drift is captured through per-feature missing value percentages logged at each run; an increasing trend signals upstream data degradation. Distributional drift is monitored via Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Score across clustering runs; degradation over successive runs indicates that the data distribution has shifted from the training baseline. Model performance drift is tracked via accuracy, F1 Score, and ROC-AUC per classification run; declining trends produce quality alerts.

B. Planned Automated Monitoring

The planned monitoring container will implement active statistical drift detection. Feature distributions from current input batches will be compared against the original training baseline using Kolmogorov-Smirnov tests for continuous features and Chi-squared tests for categorical features. A p-value below 0.05 on any monitored feature sustained over three consecutive evaluation cycles triggers a drift alert. A 7-day rolling accuracy window is maintained; when rolling accuracy falls below the configured threshold, the container automatically initiates a full pipeline re-execution and notifies operators.

C. Model Version Management

All promoted models are versioned through the MLflow Model Registry. The standard promotion workflow enforces quality gates at each transition: Registered (None) to Staging upon validation test passage; to Production upon integration test passage; to Archived upon the next promotion. This guarantees that the prior Production model is always preserved and instantly restorable through a single API call.

IX. Limitations

Sequential Execution: All six pipeline stages run in strict sequence. Stages 3 (EDA) and 4 (clustering) are computationally independent and could be parallelized, potentially reducing total pipeline time by 20–30%.

Local Artifact Storage: Artifacts are stored on the local filesystem. Production deployments require migration to distributed object storage such as Amazon S3, Google Cloud Storage, or Azure Blob Storage for durability and multi-machine access.

Batch-Only Processing: The pipeline ingests data from static CSV files only. Real-time applications require integration with a streaming platform such as Apache Kafka for continuous data ingestion.

Static Hyperparameter Configuration: Model hyperparameters are fixed at configuration time. Integration with an optimization framework such as Optuna would enable adaptive search on each retraining cycle.

Single-Machine Execution: The implementation is designed for a single compute host. Processing datasets at the scale of hundreds of millions of records would require distributed training via Spark MLlib or Dask.

X. Conclusion

This paper introduced AutoMLOps, a fully automated, modular, open-source MLOps pipeline that removes manual intervention from every stage of the machine learning lifecycle. The system delivers five concrete outcomes: full end-to-end pipeline automation through a single command completing all six stages in approximately 141 seconds; comprehensive experiment tracking with MLflow automatically logging over 40 parameters, 50 metrics, and 30 artifacts; production-quality modeling achieving 92.51% classification accuracy and a ROC-AUC of 0.9642;

monitoring-ready infrastructure where drift indicators are systematically tracked at every run; and a fully designed four-container Docker architecture ready for immediate implementation.

The system's central design value is generalizability. The pipeline makes no domain-specific assumptions; any practitioner working with structured tabular data can configure and execute the complete automated ML lifecycle—from raw ingestion through trained, tracked, versioned, and deployment-ready model—using only open-source tooling on commodity hardware. When Docker containerization is complete, a single command will launch the entire ML system across training, experiment tracking, real-time inference, and automated monitoring on any machine in any environment without provider dependency.

ACKNOWLEDGMENTS

The author expresses sincere gratitude to the faculty of the Department of Information Technology at CSPIT, CHARUSAT for their guidance, support, and the computational resources that made this work possible.

REFERENCES

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden Technical Debt in Machine Learning Systems," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015.

[2] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the Machine Learning Lifecycle with MLflow," IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[3] R. S. Olson, N. Bartley, R. J. Urbanowicz, and J. H. Moore, "Evaluation of a Tree-based Pipeline Optimization Tool for Automating Data Science," in Proc. GECCO, 2016.

[4] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and Robust Automated Machine Learning," in Advances in Neural Information Processing Systems, vol. 28, 2015.

[5] C. Boettiger, "An introduction to Docker for reproducible research," ACM SIGOPS Operating Systems Review, vol. 49, no. 1, pp. 71–79, 2015.